Integrated Business Requirements & Use Case Document

SProj 2026–27 | Final-Year CS Projects

A natural-language interface for LUMS students, faculty, and staff to initiate and complete university processes.

Group name	Height Boosters
Group number	4
Project name	LUMS Process Orchestrator
Team members and IDs	Muhammad Yahya Rahim (27100218), Muhammad Hussain Baqar (27100127), Saad Jamshaid Khan (27100159), Muhammad Ammar Anees (27100077), Nausherwan Khan Arzani (27100384)
Supervisor	Dr. Ali Khawaja
Version / date	v0.1 — September 20, 2026

1. Executive Summary

The LUMS Process Orchestrator is a natural-language interface for LUMS students, faculty, and staff to initiate and complete university processes. Students, faculty, and staff at LUMS currently manage academic and administrative processes — course enrollment, approvals, and status follow-up — through disconnected channels: Zambeel, LMS, email, and in-person visits. Requests are routinely lost in email threads, and no shared system gives a requester or an approver visibility into where a request currently stands. This project proposes a natural-language interface that sits in front of Zambeel and LMS, allowing a student, instructor, or staff member to state a goal in plain English — for example, force enrollment in a course — and have the system interpret that goal, retrieve the applicable policy, verify eligibility, route the request to the correct human approver, execute the authorized action, and track it to completion. Every consequential action requires explicit human approval and is recorded in a complete audit trail; the system never writes to a university system without a valid, approved action. The first prototype targets the CS department and centers on the force-enrollment workflow as a generalizable proof of the underlying orchestration pattern, with prerequisite/seat/timetable checks and case-status tracking as secondary capabilities. The expected value is reduced manual routing effort for students and staff, faster and more transparent approvals, and a reusable workflow engine that LUMS can extend to further departments and processes without rebuilding it for each one.

2. Problem, Opportunity, and Objectives

2.1 Current Situation and Pain Points

- Requests are spread across Zambeel, LMS, Outlook, and departmental offices, with no single owner or system tracking a request end to end.
- Students and staff frequently do not know which office or person is responsible for a given request.
- Status updates depend on manual email follow-up; long-running requests have poor visibility.
- Instructors, coordinators, and Registrar's Office (RO) staff repeatedly answer similar questions and manually transfer information between systems and emails.
- Deadlines and requirements are often buried in LMS announcements or email threads and can be missed.

2.2 Opportunity

A natural-language orchestration layer placed in front of Zambeel and LMS — rather than replacing either system — can interpret a plain-language goal, apply the correct policy and approval chain, and execute or track the resulting action, without requiring LUMS to change its systems of record.

2.3 Project Objectives

ID	Objective
O-01	Deliver a working, end-to-end force-enrollment workflow — submission, policy check, approval, execution, notification — against a mock Zambeel environment.
O-02	Record zero unauthorized (non-token-approved) writes across all test executions.
O-03	Provide a complete, retrievable audit record for 100% of processed cases.
O-04	Demonstrate that the workflow engine is generalizable by defining at least one additional workflow (e.g., TA-ship/DCW approval) through configuration — steps and approvers — without changes to core orchestration code.

3. Stakeholders and Users

Stakeholder	Role	Interest / Current pain point
Student	Primary requester	Wants a single place to request and track academic actions instead of chasing approvals over email.
Instructor / HOD	Approver	Currently approves requests over email with no structured record; wants a clear, low-effort approval action.
Department Coordinator	Process owner / routing	Manually routes and tracks departmental requests; wants reduced repetitive coordination work.
Registrar's Office (RO) — Dr. Shafay Shumail	System-of-record and policy authority	Owns the actual academic process and data; has prior concerns about outside changes to university systems and cares about improved student information access.
IST / IT — Kashif Mahmood	Data custodian	Maintains Zambeel/PeopleSoft infrastructure; is the technical gatekeeper but not the process decision-maker.
Project Supervisor — Dr. Ali Khawaja	Technical sponsor and evaluator	Requires evidence-based, quantified engineering decisions and a generalizable workflow design.

> *Structured interviews with a student representative, an instructor, a department coordinator, and RO staff were instructed by the supervisor on September 19, 2026, and are in progress at the time of this submission. This section reflects the team's current understanding and will be revised with direct stakeholder input as interviews are completed.*

4. Scope and Assumptions

4.1 In Scope — Phase 1 Pilot

- CS department only, as the initial pilot before expansion to other departments.
- Force-enrollment workflow, end to end (primary MVP workflow).
- Prerequisite, seat, and timetable-clash checks, and case-status tracking (secondary).
- Mock Zambeel adapter and synthetic/anonymized data until institutional access is confirmed.
- Single-orchestrator architecture with bounded, auditable tools rather than independent specialist agents.
- Policy-grounded retrieval (RAG) for the applicable enrollment policy, with citation of the specific section used.
- Role-based access control for student, instructor/HOD, and RO roles.

4.2 Out of Scope — This Phase

- LMS integration, and departments other than CS.
- Finance, hostel, events/CCA, and graduation-clearance workflows.
- TA-ship/DCW approval and academic advising — named by the supervisor as core use cases and planned for the next phase, not abandoned.
- Any autonomous system action taken without explicit human approval.
- Implementation of identity-federation internals (SAML token handling, certificate exchange, encryption) — per supervisor guidance, the system calls documented PeopleSoft APIs and relies on LUMS's existing identity infrastructure rather than reimplementing it.

4.3 Assumptions

- LUMS's Zambeel is an Oracle PeopleSoft (Campus Solutions) deployment, with authentication federated through LUMS's identity provider (Microsoft Entra / Active Directory) using SAML-based federated identity.
- Production integration requires access to PeopleSoft's Integration Broker REST services or Component Interfaces, to be requested through the Registrar's Office and IST.
- Zambeel's integration endpoint is reachable only from the LUMS network; institutional network access (VPN or peered connection) or an in-network gateway will be required for any real integration.
- If real system or data access is not granted during the prototype phase, synthetic data and a mocked API will be used and clearly labeled as such throughout.

5. Business Requirements

ID	Description	Verification
BR-01	Reduce manual, email-based routing and coordination for pilot academic requests.	Compare steps/emails required before vs. after, on a sample of pilot cases.
BR-02	Provide an auditable, traceable record of every request and decision.	Audit log inspection covering 100% of processed cases.
BR-03	Ensure no consequential action is executed without appropriate human approval.	Test suite confirming zero unauthorized writes.
BR-04	Give requesters and approvers real-time visibility into request status.	User can query status of any open case without contacting staff.
BR-05	Establish a workflow architecture that generalizes to additional processes and departments.	A second workflow type is defined via configuration without core code changes.

6. Functional Requirements

ID	Description	Related BR	Priority
FR-01	The system shall allow a student to submit a force-enrollment request in natural language.	BR-01, BR-04	Must
FR-02	The system shall extract the student's intent and the relevant course and student identifiers from the request.	BR-01	Must
FR-03	The system shall retrieve and cite the specific enrollment policy section applicable to the request before proceeding.	BR-03	Must
FR-04	The system shall check prerequisites, seat availability, and timetable conflicts through read-only tools before allowing submission.	BR-03	Must
FR-05	The system shall route a validated request to the designated instructor or HOD for approval.	BR-03	Must
FR-06	The system shall generate a signed, single-use approval token upon instructor/HOD approval; no write action shall proceed without it.	BR-03	Must
FR-07	The system shall execute the enrollment update only via a bounded tool call authenticated by a valid, unexpired approval token.	BR-01, BR-04	Must
FR-08	The system shall automatically notify the student of approval, rejection, or a request for missing information.	BR-04	Must
FR-09	The system shall allow a student or instructor to query the live status of any submitted case at any time.	BR-04	Must
FR-10	The system shall record every action, decision, actor, and timestamp in an immutable audit log.	BR-02	Must
FR-11	The system shall represent each workflow as a configurable definition of steps and approver roles, rather than workflow-specific hardcoded logic.	BR-05	Should

7. Non-Functional Requirements

ID	Category	Requirement	Target / Measure
NFR-01	Security	All writes to Zambeel (or its mock) shall require a valid signed approval token.	Zero unauthorized writes across all tests.
NFR-02	Security	The system shall connect only via the documented hostname with TLS certificate validation and shall never disable certificate verification.	100% of calls use zambeel.lums.edu.pk with certificate validation on.
NFR-03	Reliability	Duplicate submissions of the same request shall not result in duplicate execution.	Idempotency verified under repeated/duplicate submission tests.
NFR-04	Accuracy	The system shall correctly identify the intended workflow for a given request.	≥90% routing accuracy on the test set.
NFR-05	Auditability	Every consequential action shall have a complete, retrievable audit record.	100% coverage on processed cases.
NFR-06	Privacy	A student's data shall never be retrievable through another student's query.	Zero cross-student data leaks under adversarial test prompts.
NFR-07	Groundedness	Every policy-based recommendation shall cite the specific policy section and version relied upon.	100% of policy-based responses carry a citation.
NFR-08	Scalability / Ops	Containerization and basic observability (usage, throughput) shall be addressed as the system matures.	Documented in Phase 2 architecture; not required for Phase 1 pilot.

8. Use Case Catalogue

UC ID	Name	Primary actor	Status
UC-01	Force Enrollment	Student, Instructor/HOD	Primary — Phase 1
UC-02	Track Request Status	Student, Instructor	Secondary — Phase 1
UC-03	Prerequisite / Seat / Clash Check	Student	Secondary — Phase 1 (embedded in UC-01)
UC-04	TA-ship / DCW Approval	Student, Instructor/HOD	Planned — Phase 2
UC-05	Academic Advising Query	Student, Advisor	Planned — Phase 2

9. Detailed Use Cases

UC-01 — Force Enrollment

Primary actor(s)
Student (requester); Instructor or HOD (approver); Registrar's Office system (executor, via adapter).

Goal / brief description
Allow a student to request force enrollment in a course and have the request validated, approved, executed, and tracked without

manual email coordination.

Trigger
Student submits a natural-language request, e.g. “Force enroll me in CS 400.”

Preconditions
Student is authenticated; course exists in the current offering catalogue; enrollment policy for force enrollment is loaded and versioned in the policy store.

Main success flow

1. Student states the request in natural language.
2. Orchestrator extracts intent, student ID, and course identifier.
3. Orchestrator retrieves the applicable force-enrollment policy section and cites it.
4. Orchestrator checks prerequisites, seat availability, and timetable conflicts via read-only tools.
5. Student confirms intent to proceed; a workflow case is created.
6. System routes an approval request to the designated instructor/HOD.
7. Instructor/HOD approves the request.
8. System generates a signed, single-use approval token.
9. System executes the enrollment update via the bounded tool, authenticated by the token.
10. Student is notified of the outcome; the case and all actions are recorded in the audit log.

Alternative / exception flows

- Prerequisite, seat, or clash check fails → system explains the failure and does not proceed to approval.
- Instructor/HOD rejects the request → system explains who decided, the reason (if given), and whether another valid process exists; the request is not silently retried.
- Approval token is missing, expired, or invalid → execution is blocked and the case is flagged for review.
- Duplicate submission of an already-processed request → system detects duplication via idempotency check and returns the existing case status instead of re-executing.

Postconditions
Enrollment record is updated only on approval and successful execution; the student and instructor have a visible, final case status; a complete audit record exists regardless of outcome.

Business rules / data involved
Force-enrollment policy (versioned); student enrollment and prerequisite history; course seat count and timetable; instructor/HOD approval authority scoped to their own courses.

Acceptance criteria
A test request with valid eligibility and instructor approval results in a completed enrollment, a notification, and a full audit trail; a request with failed eligibility or without a valid approval token never results in a write to the (mock) Zambeel system.

UC-02 — Track Request Status

Primary actor(s)
Student or Instructor.

Goal / brief description
Allow a requester or approver to check the current status of a previously submitted case without emailing an office.

Trigger
User asks, e.g. “What's the status of my CS 400 enrollment request?”

Preconditions
User is authenticated; a case exists that the user is authorized to view.

Main success flow

1. User asks for the status of a case, by reference or by description.
2. System identifies the matching case, scoped to what the requesting user is authorized to see.
3. System returns the current stage (e.g., awaiting approval, approved, executed, rejected) and the next expected step, if any.

Alternative / exception flows
No matching case found → system says so plainly rather than guessing; user requests a case they are not authorized to view → system denies the query rather than disclosing any detail.

Postconditions
User receives an accurate, access-controlled status; the query itself is logged.

Business rules / data involved
Workflow case state; role-based access control on case visibility.

Acceptance criteria
A user can retrieve the correct status of their own case in a single query; a user cannot retrieve another user's case under any tested prompt.

UC-03 — Prerequisite / Seat / Clash Check

Primary actor(s)
Student; invoked directly by the student, or internally by another use case (e.g., UC-01) as a sub-check.

Goal / brief description
Determine whether a student is eligible to enroll in a given course by checking prerequisites, seat availability, and timetable conflicts, independent of whether an enrollment request is actually submitted.

Trigger
Invoked automatically as part of UC-01, or on demand when a student asks, e.g. “Am I eligible for CS 400?”

Preconditions
Student is authenticated; the course exists in the current offering catalogue; the student's academic record and current timetable are accessible.

Main success flow

1. Requesting flow supplies the student ID and course identifier.
2. System retrieves the student's completed and in-progress courses.
3. System checks the course's prerequisite requirements against the student's record.
4. System checks current seat availability for the course offering.
5. System checks the student's existing timetable for conflicts with the course's scheduled slot.
6. System returns a combined eligibility result, with the specific reason for any failing check.

Alternative / exception flows

- Prerequisite not met → system reports which specific prerequisite is missing.
- No seats available → system reports the current seat count and waitlist status, if any.
- Timetable clash → system reports the conflicting course and time slot.
- Student record incomplete or unavailable → system reports that eligibility cannot be determined and does not default to a pass.

Postconditions
The requesting flow receives a definitive eligibility result; this check is read-only and modifies no system record.

Business rules / data involved
Course prerequisite rules; current enrollment/seat counts; the student's academic history and timetable.

Acceptance criteria
For a known eligible student/course pair, all three checks pass with no write action; for a known ineligible pair, the specific failing check and reason are correctly identified.

UC-04 — TA-ship / DCW Approval

Primary actor(s)
Student (applicant); Instructor/Course Coordinator (first-level approver); Department/DCW (final confirmation).

Goal / brief description
Allow a student to apply for a TA-ship or DCW position for a course and have the application routed through instructor and departmental approval, instead of manual email coordination.

Trigger
Student submits a request, e.g. “I want to apply for a TA position for CS 200.”

Preconditions
Student is authenticated and meets baseline eligibility (e.g., minimum grade in the course, academic standing); a TA/DCW vacancy exists for the relevant course and term.

Main success flow

1. Student states intent to apply for a TA/DCW position for a specific course.
2. System retrieves baseline eligibility criteria and checks them against the student's record.
3. Student confirms and submits the application; a workflow case is created.
4. System routes the application to the course instructor for review.
5. Instructor approves or shortlists the applicant.
6. System routes the instructor's decision to the department/DCW for final confirmation.
7. On confirmation, the system records the assignment and notifies the student of the outcome.
8. All decisions and timestamps are recorded in the audit log.

Alternative / exception flows

- Baseline eligibility not met → system informs the student and does not forward the application.
- Instructor rejects the applicant → student is notified of the decision; no further routing occurs.

- Department/DCW does not confirm within the expected timeframe → case remains pending and stays visible via UC-02.

Postconditions
The TA/DCW assignment record reflects the outcome; the student and instructor have a final, visible status; a complete audit record exists.

Business rules / data involved
TA/DCW eligibility policy; course TA vacancy data; instructor and departmental approval authority.

Acceptance criteria
An eligible test application, approved at both instructor and department level, results in a recorded assignment and student notification; an ineligible application is stopped before instructor routing.

> *Preliminary design. The exact TA/DCW eligibility policy and approval chain have not yet been confirmed with the department; this use case will be revised once validated, consistent with its Phase 2 status in Sections 4 and 10.*

UC-05 — Academic Advising Query

Primary actor(s)
Student; Academic Advisor (as information owner, not an active approver in this use case).

Goal / brief description
Allow a student to ask a plain-language academic-planning question — degree progress, course recommendations — and receive a grounded, policy-cited answer, with no enrollment or record-changing action taken.

Trigger
Student asks, e.g. “What do I still need to graduate?” or “Which electives satisfy my Humanities requirement?”

Preconditions
Student is authenticated; the student's academic record and the relevant degree-requirement policy are accessible.

Main success flow

1. Student asks an advising question in natural language.
2. System retrieves the student's academic record and the relevant degree-requirement policy.
3. System compares completed and in-progress courses against requirements and identifies any gaps.
4. System responds with the specific missing requirements and, where applicable, eligible courses that would satisfy them, citing the policy source used.
5. System logs the query and response for audit purposes.

Alternative / exception flows

- Requirement data or policy is ambiguous or conflicting → system states this explicitly rather than guessing, and suggests escalation to a human advisor.
- Question falls outside the system's supported advising scope → system says so plainly and does not fabricate an answer.

Postconditions
Student receives an accurate, cited advising response; no enrollment or record change occurs, consistent with the informational autonomy level defined for this use case.

Business rules / data involved
Degree/program requirement policy; the student's academic transcript record.

Acceptance criteria
For a test student with known missing requirements, the system correctly identifies the gap and cites the applicable policy section; the system takes no write action under any advising query.

> *Preliminary design. The intended depth of advising (e.g., whether it extends to career or funding guidance, per earlier project drafts) has not yet been confirmed with an academic advisor interview.*

10. Risks, Open Questions, and Decisions

10.1 Risks and Open Questions

- No confirmed PeopleSoft Integration Broker / REST API access has been granted by LUMS IT or the Registrar's Office at the time of this submission.
- Zambeel's integration endpoint resolves to a private network address; institutional network access (VPN or peered connection) has not yet been arranged for the team's environment.
- Federated identity (Microsoft Entra / SAML) has been confirmed conceptually but not tested against the team's application; per supervisor guidance, deep implementation of federation internals is out of scope for this phase.
- Stakeholder interviews (student representatives, an instructor, department coordinator, and RO staff) are in progress and not yet complete; Sections 3 and 5 may be refined once conducted.

- The supervisor's September 8 guidance named force enrollment, TA-ship/DCW approval, and academic advising as core use cases; this submission prioritizes force enrollment as the Phase 1 pilot and schedules the other two for Phase 2, pending supervisor confirmation that this sequencing is acceptable.
- Bureaucratic and slow LUMS institutional procedures may delay approvals, data-access grants, and other decisions the project timeline depends on.
- Availability of stakeholders (students, instructors, department coordinators, RO staff) for interviews and validation may be limited, which could slow requirements refinement and use-case validation.

10.2 Decisions Made

- Adopt a single-orchestrator, bounded-tool architecture rather than a multi-specialist-agent design, to minimize the number of independent reasoning surfaces that must be secured and audited.
- Use a mock Zambeel adapter with synthetic/anonymized data for the prototype until real institutional access is confirmed.
- Require a signed, single-use approval token for every write action; no write proceeds on LLM judgment alone.
- Pilot the system within the CS department before expanding to other departments, per supervisor direction.

11. Traceability Matrix

FR	Related BR	Covered by UC	Notes
FR-01	BR-01, BR-04	UC-01	
FR-02	BR-01	UC-01	
FR-03	BR-03	UC-01	
FR-04	BR-03	UC-01, UC-03	
FR-05	BR-03	UC-01	
FR-06	BR-03	UC-01	
FR-07	BR-01, BR-04	UC-01	
FR-08	BR-04	UC-01, UC-02	
FR-09	BR-04	UC-02	
FR-10	BR-02	UC-01, UC-02	
FR-11	BR-05	—	Cross-cutting architectural requirement; validated via Phase 2 design review, not a single use case.

12. Approval / Review

Name	Role	Signature	Date
Muhammad Yahya Rahim	Team member		
Muhammad Hussain Baqar	Team member		
Saad Jamshaid Khan	Team member		
Muhammad Ammar Anees	Team member		
Nausherwan Khan Arzani	Team member		
Dr. Ali Khawaja	Supervisor		